

# quantfit — plan step deep dive

Milestone 1: artifact schemas, VRAM budget math, greedy solver

Branch `feat/plan-solver-schemas-budget` · PR #1 · 2026-07-27

# Context

- Pipeline: **scan** → **plan** → **pack** (ADR-0002), connected by versioned JSON artifacts.
- Target: Nemotron Super 49B serving on a 24 GiB RTX 4090 via vLLM (ADR-0003, ADR-0004).
- Plan built first, deliberately:
  - ADR-0005: plan must be pure Python (stdlib + typer, **no torch**) → fully CPU-testable, cheap to iterate.
  - Its budget math turns the "does this even fit" question into arithmetic before the expensive GPU work starts.
- Scan (torch, 98 GB model) develops against these schemas later.

# Architecture

Three new modules, one extended:

- `artifacts.py` — SensitivityMap / Recipe dataclasses; validating `from_dict` / `to_dict` / `load` / `save`.
- `budget.py` — size parsing, KV-cache math, weight-budget ledger.
- `solver.py` — greedy damage-per-byte solver (ADR-0007).
- `cli.py` — new `quantfit budget` and `quantfit plan` commands.

Dependency shape: `solver` → `artifacts`; `budget` standalone;  
`cli` → all three. Nothing imports torch.

# Contract: sensitivity map (scan → plan)

```
{
  "quantfit_schema": 1,
  "model_id": "nvidia/Llama-3_3-Nemotron-Super-49B-v1_5",
  "scan": { "metric": "kl_divergence", "precisions": [8, 4, 3, 2], ... },
  "groups": [{
    "name": "model.layers.0.self_attn",
    "bytes_fp16": 1000000000,
    "sensitivity": { "8": 0.0001, "4": 0.0042, "2": 0.417 }
  }]
}
```

- v1 rule: every group's sensitivity keys must equal `scan.precisions`.
- Validation errors carry JSON paths:  
`$.groups[3].sensitivity: key "4x" is not an integer precision`.
- Booleans explicitly rejected where numbers are expected  
(JSON `true` is a valid Python int)

# Contract: recipe (plan → pack)

```
{
  "plan": {
    "weight_budget_bytes": 21474836480,
    "predicted_damage": 0.0871,
    "solver": "greedy-damage-per-byte",
    "pins": { "model.layers.0.*": 8 },
    "format_overhead": 0.05,
    "trace": [{ "step": 1, "group": "...", "from_bits": 8,
                "to_bits": 4, "ratio": 1.9e-12 }]
  },
  "assignments": [{ "group": "...", "bits": 8, "bytes": 500000000,
                    "damage": 0.0001 }]
}
```

- `format_overhead` + `trace` were schema additions found during design: without them a recipe isn't reproducible and has no explanation.
- Replaying `trace` from all-highest-precision reproduces the

# Budget math — heterogeneity matters

Real target config ( `config.json` , DeciLM/NAS):

- 80 blocks; **31 have attention deleted** ( `no_op: true` ).
- 49 attention blocks, GQA group size 8  $\rightarrow$  8 KV heads  $\times$  head\_dim 128.
- FFN widths vary 10 $\times$  ( `ffn_mult` 0.5  $\rightarrow$  5.25).

Consequences baked into `budget.py` :

- KV cost = **sum over attention layers**, never `layers  $\times$  constant` .
- `ModelShape.from_config_json` parses both DeciLM `block_configs` and standard llama configs.
- Hand-computed anchor test:  $2 \times 49 \times 8 \times 128 \times 2 = \mathbf{200,704}$  bytes/token at fp16.

# Measured results

```
quantfit budget --model-config <nemotron> --vram 24GiB --context 16384
```

|                      | fp16 KV          | fp8 KV           |
|----------------------|------------------|------------------|
| VRAM total           | 24.00 GiB        | 24.00 GiB        |
| – KV cache @16k      | 3.06 GiB         | 1.53 GiB         |
| – runtime overhead   | 2.00 GiB         | 2.00 GiB         |
| <b>weight budget</b> | <b>18.94 GiB</b> | <b>20.47 GiB</b> |

→ ~3.3–3.5 average bits/parameter over ~49B params.

Better than the docs' original ~3.2 estimate; still under vLLM's 4-bit kernel floor (see risks slide).

# Solver (ADR-0007, now Accepted)

Multiple-choice knapsack, greedy by damage-per-byte:

1. Start every group at highest scanned precision (or its pin).
2. Precheck: minimum achievable total vs budget → `InfeasibleBudgetError` with exact gap in bytes.
3. While over budget: apply the move minimizing `(damage_delta / bytes_freed, group_name, smallest_step)`.



# Solver — design points

- Moves consider **all** lower precisions → non-convex damage curves get direct multi-step jumps (8 → 2 without stopping at 3).
- Selection key is a total order → deterministic, input-order invariant (tested under shuffle + pytest-randomly).
- Pins: `fnmatchcase` globs; zero-match = hard error; later overrides earlier; pinned groups never move.
- Integer byte math ( `ceil` once per size) — no float accumulation to threaten determinism.

# Why greedy (and what was rejected)

- Instance is small (hundreds of groups  $\times$   $\sim 4$  precisions) — exact DP and ILP are both feasible.
- Greedy chosen first because its downgrade sequence is the explanation trace — "why is this group 4-bit?" has a literal answer.
- Damage model is itself approximate (marginal additivity, ADR-0006 still Proposed) — an exact optimum of an approximate objective is still approximate.
- `--solver exact` planned later as a check on greedy's gap.

# Verification

- **88 tests, 99% line coverage** (gate: 90%).
- Determinism: repeated runs identical; shuffled group order → identical assignments and trace.
- Budget safety: property-style sweep asserts  
`predicted_total ≤ budget` across the feasible range.
- Full gates green: ruff, ruff format, ty, pytest, docvet (0 findings, 100% docstring coverage).
- Deliberately not covered: no GPU tests exist yet (`gpu` marker reserved); scan pipeline is still a stub.

# Risks and open questions

- **ADR-0003 vs ADR-0004 tension (now measured):** budget forces ~3.3–3.5 avg bits; vLLM kernels floor at 4-bit (no 2/3-bit).
  - Paths: friendlier measured overheads · contribute sub-4-bit kernel · GGUF backend for the benchmark · change target model.
  - Deferred to its own ADR before `scan` fixes candidate precisions.
- Marginal-additivity assumption (ADR-0006, Proposed) — mitigation: whole-recipe validation pass after plan.
- `format_overhead` (5%) and runtime overhead (2 GiB) are planning constants until measured for real.

# Next milestone

- **Scan harness:** torch + offload behind `quantfit[scan]` extra (ADR-0005 keeps base install clean).
- Streams layer groups to GPU; 124 GB system RAM holds the bf16 reference; KL divergence per (group × precision).
- Its output feeds today's `plan` unchanged — the artifact contract is already enforced from both sides.
- Before that: resolve the 4-bit-floor tension ADR.

# Links

- PR #1: `github.com/Alberto-Codes/quantfit/pull/1`
- ADRs: `docs/adr/` — 0007 promoted to Accepted this milestone.
- Schemas: `docs/reference/sensitivity-map.md`, `docs/reference/recipe.md` (both promoted sketch → draft).
- Budget math: `docs/explanation/vram-budget.md` (promoted → stable, real numbers).
- Glossary: `docs/reference/glossary.md` — one term per concept.